AI10 · Agentes en producción

MCP · Orquestación · Observabilidad · Evals · Seguridad

AI Surfer · N3 AI Engineering

Por qué este módulo

- Ofertas de *AI Engineer* (oct. 2026): **MCP**, agentes, evaluación de agentes, *tracing*, seguridad
- «Funciona en mi demo» ≠ «puedo demostrar que sigue funcionando»
- Al terminar: un repositorio con **servidor MCP propio + agente + evals en CI**

Mapa del módulo

1. Qué es MCP
2. Escribir un servidor
3. Probarlo sin red ni clave
4. El agente: política y confirmación
5. Claude: bucle, *tool runner*, conector
6. Multiagente (y cuándo no)
7. A2A
8. Observabilidad
9. Evals de trayectorias en CI
10. Seguridad

10.1 · El problema: N×M integraciones

- Cada agente reescribe «consultar pedido»: descripción, esquema, errores, auth
- **MCP**: estándar abierto → N + M piezas
- **Servidor**: expone capacidades · **Cliente**: las **descubre** en ejecución (`tools/list`)

Capacidad	La usa…	Ejemplo
Herramienta	el **modelo**	`crear_ticket`
Recurso	la **aplicación**	`politicas://garantia`
Prompt	la **persona**	`triage_soporte`

10.1 · Transportes

- **stdio**: servidor = subproceso local. Simple, para desarrollo
- **Streamable HTTP**: servidor remoto, varios clientes, OAuth
- Debajo: **JSON-RPC 2.0** → `initialize`, `tools/list`, `tools/call`

> MCP no mejora tus herramientas ni te protege solo.

10.2 · Un servidor en ~60 líneas

```python
from mcp.server.mcpserver import MCPServer   # mcp 1.x: FastMCP
mcp = MCPServer("tiendasur", instructions="...")

@mcp.tool(annotations=SOLO_LECTURA)
def consultar_pedido(pedido_id: str, email: str) -> dict:
    """Devuelve el estado de un pedido... No modifica nada."""
```

- Tipos → **esquema**; docstring → **descripción**
- Descripción + esquema = la «interfaz de usuario» del modelo

10.2 · Diseño de herramientas

- Pocas, bien diferenciadas, descripción para quien no conoce tu sistema
- `Literal["baja", "normal"]`: el servidor rechaza lo no permitido
- `ToolError("...")`: el modelo ve **tu** mensaje
- Mismo error para «no existe» y «no es tuyo»
- **Idempotencia** en lo que tiene efectos
- Anotaciones = **pistas**, no garantías

10.3 · Probar sin red ni clave

- **En memoria**: `Client(servidor)` → tests rápidos y deterministas
- **stdio**: subproceso real → prueba de integración
- El subproceso **no hereda tu entorno** (ni tu clave)
- JSON-RPC a mano: tres mensajes
- MCP Inspector para verlo en visual

10.4 · El agente: lo que NO es el modelo

```
Politica(permitidas=..., con_efectos=..., confirmar=..., max_pasos=8)
```

1. **Allowlist** (se filtra antes y se bloquea después)
2. **Confirmación humana** — por defecto, *deniega*
3. **Límite de pasos**
4. Resultados marcados como **datos**

10.4 · Cerebro intercambiable

- `CerebroSimulado` ↔ `CerebroClaude`: misma interfaz
- CI gratis, rápido y determinista; mismas trazas con el modelo real
- API: **todas** las `tool_use` en **un** mensaje · `resp.content` completo · texto con `next(b.text ...)`

10.5 · Tres caminos con Claude

Camino	Bucle	Control
Manual	tú	total
Tool runner + `async_mcp_tool`	SDK	medio
Conector MCP (`mcp_servers` + `mcp_toolset`)	Anthropic	bajo

Con efectos o auditoría → tu bucle.

10.6 · Multiagente

- Supervisor → trabajadores con permisos distintos
- **Sí**: aislar permisos, demasiadas herramientas, paralelismo real
- **No**: tarea lineal, mucho contexto compartido, sin medir
- Coste: más viajes al modelo, más fallos posibles

Empieza con un agente.

10.7 · A2A

- MCP: agente ↔ **herramientas** (vertical)
- A2A: agente ↔ **agente de otra organización** (horizontal)
- *Agent card* publicada en una URL conocida
- Hoy: MCP basta. Comprueba la especificación vigente

10.8 · Observabilidad

```
agente.tarea
├─ llm.paso            (modelo, tokens, coste)
├─ herramienta.X       (args, error, bloqueada)
└─ llm.paso
```

- OpenTelemetry: cambias el exportador, no los spans
- **Coste y latencia por tarea**, percentiles p50/p95, presupuestos
- Cuidado con datos personales en trazas

10.9 · Evaluar trayectorias

La respuesta puede ser correcta por el **camino equivocado**.

`debe_llamar` · `orden` · `prohibidas` · `max_llamadas` · sin repetidas · sin efectos sin confirmar

- Deterministas y baratos → **cada PR**
- Con Claude → solo en **main**
- **Meta-test**: un eval que no sabe fallar no sirve

10.9 · Evals en CI

```
- run: ruff check .
- run: pytest -q           # servidor + evals, sin clave
- if: github.ref == 'refs/heads/main'
  run: EVAL_CON_CLAUDE=1 pytest tests/test_trayectorias.py
```

10.10 · Seguridad en capas

Riesgo	Defensa
Inyección en resultados	datos ≠ órdenes · allowlist · eval
Descripción envenenada / *rug pull*	huellas + auditoría
Exceso de privilegio	mínimo privilegio, validación en servidor
Acciones irreversibles	confirmación · idempotencia · límites
Fuga de secretos	entorno mínimo

El modelo no es una frontera de seguridad.

Caso práctico

Repo en GitHub con:

1. Servidor MCP propio (≥ 3 herramientas, una con efectos)
2. Agente con allowlist y confirmación
3. Trazas con coste y latencia
4. ≥ 8 casos de trayectoria + meta-test
5. CI en verde y README honesto

Resumen

Estándar (MCP) · **control** (política) · **visibilidad** (trazas) · **garantías** (evals en CI) · **defensa en capas**